CÔNG NGHỆ JAVA

BÀI 9

KẾT NỐI CƠ SỞ DỮ LIỆU

GV. Bùi Văn Công
Mobile: 0983 978 015

CÁC TRÌNH ĐIỀU KHIỂN JDBC

- Hiện nay có rất nhiều Hệ quản trị CSDL (DBMS) khác nhau như SQL Sever, MySQL, Oracle, MS Access, FoxPro,…
- Để truy cập các DBMS khác nhau từ chương trình viết bằng Java thì ta cần có các JDBC driver tương ứng.
- Hãng Sun đã đưa ra 4 loại JDBC driver.
- Tham khảo tại: http://industry.java.sun.com/products/jdbc/drivers

CÁC TRÌNH ĐIỀU KHIỂN JDBC

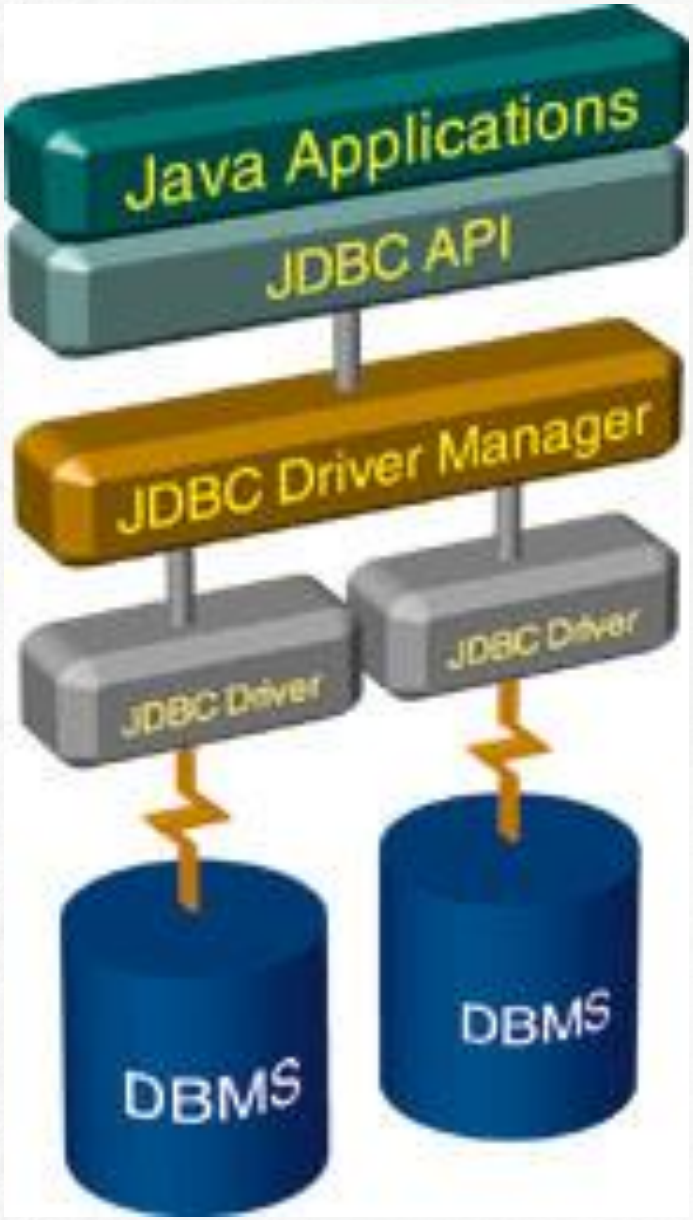
- Tương tác căn bản nhất của JDBC được liệt kê sau đây:
  - Mở một kết nối đến CSDL (Open connection).
  - Thực thi các câu lệnh SQL (Execute SQL).
  - Xử lý dữ liệu (Process result).
  - Đóng kết nối (Close conection).

CÁC TRÌNH ĐIỀU KHIỂN JDBC

- Sử dụng JDBC API, chương trình ứng dụng có thể thiết lập kết nối đến hệ quản trị CSDL, giao tiếp với CSDL, thực thi các câu lệnh thao tác, truy vấn dữ liệu và nhận kết quả trả về.

- Kiến trúc của JDBC gồm hai tầng: tầng đầu tiên là JDBC API, có nhiệm vụ chuyển các câu lệnh SQL cho bộ quản lý trình điều khiển JDBC; tầng thứ hai là các JDBC Driver API, thực hiện nhiệm vụ liên hệ với trình điều khiển của hệ quản trị CSDL cụ thể như SQL-Server, MS-Access, MySQL, IBM-DB2, Oracle,…

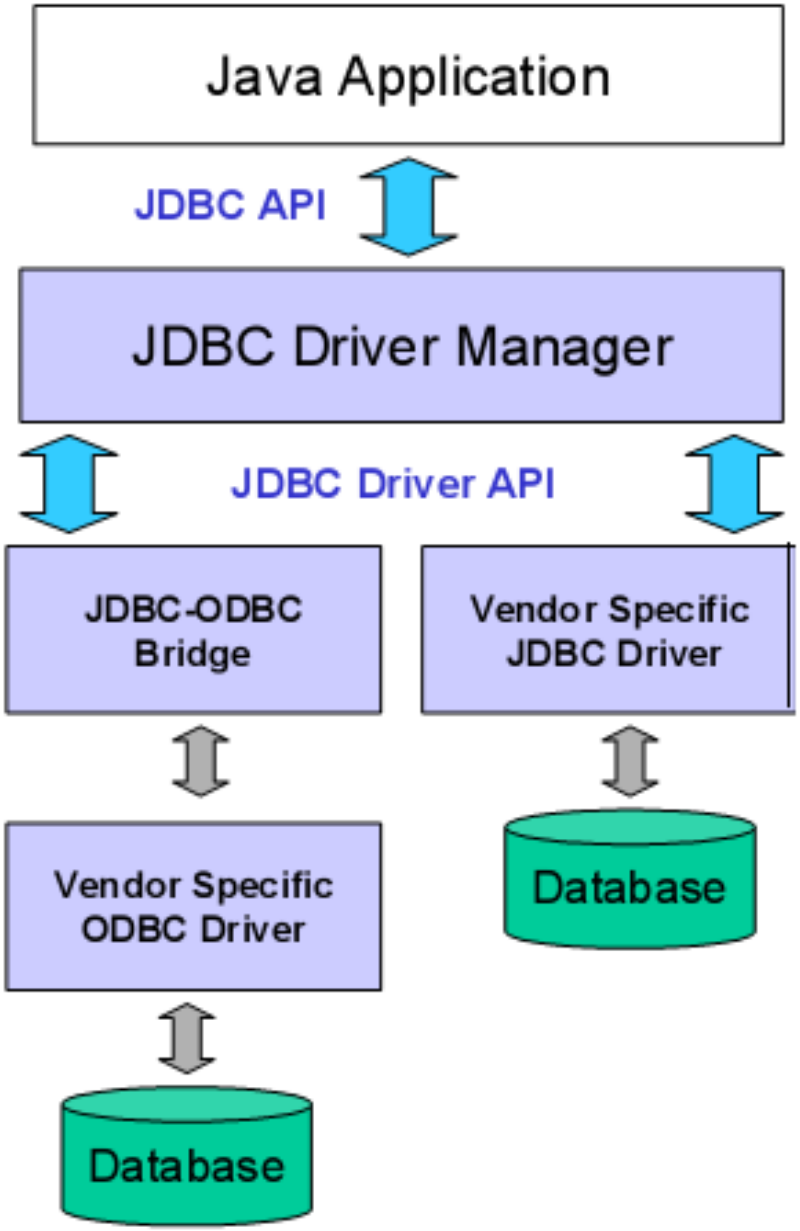
CÁC TRÌNH ĐIỀU KHIỂN JDBC



CÁC TRÌNH ĐIỀU KHIỂN JDBC

- Các trình điều khiển có nhiệm vụ là yêu cầu hệ quản trị CSDL thực hiện các câu lệnh SQL. Các trình điều khiển trong JDBC là các đoạn chương trình do nhà sản xuất hệ quản CSDL cung cấp hoặc do các nhà cung cấp thứ ba đưa ra. Các trình điều khiển JDBC được chia ra làm bốn loại:
  - Cầu nối JDBC-ODBC (Bridge)
  - Trình điều khiển thuần túy Java (Native-API).
  - Trình điều khiển JDBC-Net, Pure Java.
  - Trình điều khiển Native-Protocol, Pure Java.

CƠ CHẾ HOẠT ĐỘNG VỚI JDBC



CÁC BƯỚC LẬP TRÌNH TRUY XUẤT CSDL

- Nạp trình điều khiển.
- Thiết lập kết nối.
- Tạo đối tượng Statement
- Thực hiện vấn tin
- Xử lý kết quả trả về
- Đóng kết nối

NẠP TRÌNH ĐIỀU KHIỂN

- Sử dụng phương thức tĩnh forName() của lớp Class với tham số là tên trình điều khiển cơ sở dữ liệu.

- ***Cách dùng:***

```
try{
    Class.forName(“Database driver name”);
}
catch(ClassNotFoundException e){
    System.out.println(“Driver not found: ” +e.getMessage());
}
catch(SQLException e){
    System.out.println(“SQL Exception: ” + e.getMessage());
}
```

NẠP TRÌNH ĐIỀU KHIỂN

- Trình điều khiển của MySQL:

  Class.forName(“com.mysql.jdbc.Driver”);
- Trình điều khiển của Oracle:

  Class.forName(“oracle.jdbc.driver.OracleDriver”);
- Trình điều khiển của Sybase:

  Class.forName("com.sybase.jdbc.SybDriver");
- Trình điều khiển qua cầu nối ODBC:

  Class.forName(“sun.jdbc.odbc.JdbcOdbcDriver”);

THIẾT LẬP KẾT NỐI

- Để thiết lập kết nối ta gọi phương thức tĩnh **getConnection()** của lớp DriverManager, khi đó trả về một thể hiện của lớp Connection, theo dạng như sau:
  - **String user = “sa”**
  - **String password = “secret”**
  - **Connection con = DriverManager.getConnection(dbUrl, username, password);**
- Trong đó:
  - **dbUrl**: là chuỗi kết nối đến cơ sở dữ liệu.
  - **username** : tên người dùng đăng nhập
  - **password** : mật khẩu đăng nhập.

VÍ DỤ VỀ THIẾT LẬP KẾT NỐI

- Nạp trình điều khiển jdbc driver cần sử dụng

  Class.forName("com.mysql.jdbc.Driver");

- Tạo kết nối:

Connection con=DriverManager.getConnection(

"jdbc:mysql://localhost:3306/sinhvien","root",""“)

TẠO ĐỐI TƯỢNG Statement

- **Sử dụng đối tượng Connection để tạo đối tượng Statement để thực thi câu lệnh**

  Statement st = con.createStatement();

- **Đối tượng này có nhiệm vụ gửi các câu lệnh sql đến CSDL**
- **Cùng một đối tượng Statement có thể sử dụng cho nhiều câu lệnh sql khác nhau.**
- **Có 3 phương thức thực thi**
  - ✓ **executeQuery()**
  - ✓ **executeUpdate()**
  - ✓ **execute()**

VÍ DỤ (tạo Statement và vấn tin)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
public TestConnection() throws ClassNotFoundException,
SQLException{
//Nạp trình điều khiển
Class.forName("com.mysql.jdbc.Driver");
//Thực hiện kết nối
Connection con = DriverManager.getConnection(
"jdbc:mysql://localhost:3306/sinhvien","root","") }
```

VÍ DỤ (tạo Statement và vấn tin)

```
public ResultSet GetData(String tbName) throws
SQLException {
   ResultSet kq = null;
//Tạo đối tượng truy vấn
  Statement statement =
this.con.createStatement();
String sql = "select * from Products";
//Thực thi truy vấn, trả kết quả kq
kq=statement.executeQuery(sql);
return kq;   }
```

VÍ DỤ (tạo Statement và vấn tin)

```
public static void main(String[] args) throws
ClassNotFoundException, SQLException {
TestConnection pt = new TestConnection();
 ResultSet rs = pt.GetData("Products");
  while (rs.next()) {
     System.out.print(rs.getString(“Username")+“ “
+rs.getString(“Password")+" "+rs.getString("Number"));
     System.out.println("");
                        }
//Đóng kết nối
     pt.Close();
    }
```

TẠO BẢNG QUA JDBC

```
public void createTable(){
    String sqlQuery = "CREATE TABLE PRICELIST(NAME
                              VARCHAR(20), PRICE NUMBER);";
    if (conn != null){
          try{
                Statement st = conn.createStatement();
                st.execute(sqlQuery);
                st.close();
          }catch(SQLException e){
                e.printStackTrace();
          }
    }
}
```

NHẬP DỮ LIỆU QUA JDBC

- Sử dụng phương thức executeUpdate() của Statement để cập nhật dữ liệu qua JDBC
- executeUpdate() trả về số dòng bị ảnh hưởng sau khi câu lệnh được thực thi

Ví dụ:

```
public void insertData(String name, double val){
    String insertQuery = "INSERT INTO PRICELIST VALUES
                                        ('" + name + "'," + val + ");";
    if (conn != null){
        try{
            Statement st = conn.createStatement();
            int d = st.executeUpdate(insertQuery);
            System.out.println("So dong duoc cap nhat: " + d);
            st.close();
        }catch(SQLException e){
            e.printStackTrace();
        }
    }
}
```

ĐỌC DỮ LIỆU TỪ CSDL

- Sử dụng phương thức **executeQuery()** của Statement để đọc dữ liệu từ cơ sở dữ liệu.
- Dữ liệu đọc được sẽ được lưu trong ResultSet.
- Có thể di chuyển tới lui trong ResultSet để lấy dữ liệu ra.



VÍ DỤ

```
public void selectAll(){
	String selectQuery = "SELECT * FROM PRICELIST;";
	if (conn != null){
	try{
		Statement st =
			conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
		ResultSet.CONCUR_READ_ONLY);
		ResultSet rs = st.executeQuery(selectQuery);
		System.out.println("NAME\t\tPRICE");

		while (rs.next()){
			String name = rs.getString("NAME");
			double val = rs.getDouble("PRICE");
			System.out.println(name + "\t\t" + val);
		}
		st.close();
	}catch(SQLException e){
		e.printStackTrace();
	}
	}
}
```

CÁC KIỂU Resultset

- TYPE_FORWARD_ONLY
  - Con trỏ của ResultSet kiểu này chỉ được di chuyển theo một hướng từ đầu đến cuối
- TYPE_SCROLL_INSENSITIVE
  - Con trỏ có thể di chuyển tới lui tương đối với vị trí hiện tại của nó, và cũng có thể di chuyển đến một vị trí cụ thể, không bị ảnh hưởng nếu kết quả được thay đổi ở nơi khác
- TYPE_SCROLL_SENSITIVE
  - Con trỏ có thể di chuyển tới lui tương đối với vị trí hiện tại của nó, và cũng có thể di chuyển đến một vị trí cụ thể, sẽ bị ảnh hưởng nếu kết quả bị thay đổi nơi khác

CÁC CHẾ ĐỘ HOẠT ĐỘNG CỦA RESULTSET

Các chế độ hoạt động đồng thời của Resultset:

- CONCUR_READ_ONLY
  - Xác định chế độ hoạt động đồng thời, kết quả lưu trong đối tượng ResultSet không được thay đổi
- CONCUR_UPDATABLE
  - Xác nhận chế độ hoạt động đồng thời, kết quả lưu trong đối tượng ResultSet được thay đổi

CÁC PHƯƠNG THỨC CỦA RESULTSET

- next(): di chuyển con trỏ đến dòng kế, trả về true nếu có dòng kế tiếp, false nếu đến cuối ResultSet
- previous(): di chuyển con trỏ đến dòng trước
- first(): di chuyển con trỏ đến dòng đầu tiên
- last(): di chuyển con trỏ đến dòng cuối cùng
- beforeFirst(): di chuyển con trỏ đến vị trí trước dòng đầu tiên
- afterLast(): di chuyển con trỏ đến sau dòng cuối cùng
- relative(int rows): di chuyển con trỏ tương đối với vị trí hiện tại của nó với số dòng là rows
- absolute(int row): di chuyển con trỏ đến dòng thứ row

LẤY DỮ LIỆU TỪ RESULTSET

- Dùng phương thức getXXX(String colname)
  - XXX là kiểu dữ liệu được trả về
  - colname là tên của cột cần lấy dữ liệu ra
- Ví dụ:
  - String name = rs.getString("NAME");
  - double val = rs.getDouble("PRICE");

CÁC PHƯƠNG THỨC getXXX

Kiểu của SQL	Kiểu của Java	Hàm getXXX()
CHAR	String	getString()
VARCHAR	String	getString()
LONGVARCHAR	String	getString()
NUMBERIC	java.math.BigDecimal	getBigDecimal()
DECIMAL	java.math.BigDecimal	getBigDecimal()
BIT	Boolean (boolean)	getBoolean()
TINYINT	Integer (byte)	getByte()
SMALLINT	Integer (short)	getShort()
INTEGER	Integer (int)	getInt()
BIGINT	Long (long)	getLong()
REAL	Float (float)	getFloat()
FLOAT	Double (double)	getDouble()
DOUBLE	Double (double)	getDouble()
BINARY	byte[]	getBytes()
VARBINARY	byte[]	getBytes()
LONGVARBINARY	byte[]	getBytes()
DATE	java.sql.Date	getDate()
TIME	java.sql.Time	getTime()
TIMESTAMP	java.sql.Timestamp	getTimestamp()

CHỈNH SỬA DỮ LIỆU BẰNG RESULTSET

- Dữ liệu của một bảng trong CSDL có thể được chỉnh sửa bằng cách sử dụng câu lệnh SQL UPDATE
- Ta cũng có thể chỉnh sửa dữ liệu trên các hàng của một bảng từ ResultSet
- Để làm được điều này, ResultSet phải được đặt ở chế độ CONCUR_UPDATABLE
- Sử dụng các phương thức updateXXX() để chỉnh sửa dữ liệu trên ResultSet
  - XXX là kiểu dữ liệu của cột cần được sửa

VÍ DỤ CHỈNH SỬA DỮ LIỆU

```
public void updateTable(){
    String selectQuery = "SELECT * FROM PRICELIST;";
    if (conn != null){
        try{
            Statement st =
        conn.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
                    ResultSet.CONCUR_UPDATABLE);
            ResultSet rs = st.executeQuery(selectQuery);
            rs.next();
            rs.updateDouble("PRICE", 1.5);
            rs.updateRow();
            st.close();
        }catch(SQLException e){
            e.printStackTrace();
        }
    }
}
```

SỬ DỤNG PreparedStatement

- Có những trường hợp ta cần thực hiện nhiều câu lệnh SQL có cấu trúc tương tự nhau, chỉ có giá trị là thay đổi
- PreparedStatement có thể được sử dụng để soạn trước câu lệnh có sẵn cấu trúc cần thiết
- Giá trị sẽ được đưa vào như những đối số khi câu lệnh được thực thi

VÍ DỤ PreparedStatement

```
public void prepareStatement(){
    String insertQuery = "INSERT INTO PRICELIST VALUES(?,?);";
    if (conn != null){
    try{
            PreparedStatement prest = conn.prepareStatement(insertQuery);
            prest.setString(1, "Biscuit");
            prest.setDouble(2, 1.2);
            prest.executeUpdate();
            prest.setString(1, "Pen");
            prest.setDouble(2, 0.5);
            prest.executeUpdate();
            prest.close();
    }catch(SQLException e){
            e.printStackTrace();
    }
    }
}
```

SỬ DỤNG Transaction (Giao Dịch)

- Mặc định, sau khi mỗi câu lệnh SQL được thực thi qua JDBC, dữ liệu sẽ được cập nhật ngay vào CSDL
- Có những trường hợp, ta muốn dữ liệu chỉ được cập nhật vào CSDL sau khi một số câu lệnh SQL được thực hiện
  - Ví dụ: đối với trang ứng dụng bán hàng qua mạng, để CSDL được thống nhất, ta chỉ muốn lưu các dữ liệu liên quan tới một đơn đặt hàng cùng một lúc
- Một nhóm các câu lệnh như thế được gọi là một giao dịch (transaction)

CÀI ĐẶT GIAO DỊCH

- Trước hết, phải không dùng chế độ COMMIT tự động từ Connection object
  - conn.setAutoCommit(false);
- Thực hiện các câu lệnh trong một giao dịch
- Thực hiện COMMIT (lưu) CSDL
  - conn.commit();
- Nếu không cần dùng ở chế độ giao dịch nữa, ta nên trả lại chế độ COMMIT tự động
  - conn.setAutoCommit(true);

VÍ DỤ Transaction

```
public void transaction(){
    String insertQuery = "INSERT INTO PRICELIST VALUES(?,?);";
    if (conn != null){
    try{
            conn.setAutoCommit(false);
            PreparedStatement prest = conn.prepareStatement(insertQuery);
            prest.setString(1, "Biscuit");
            prest.setDouble(2, 1.2);
            prest.executeUpdate();
            prest.setString(1, "Pen");
            prest.setDouble(2, 0.5);
            prest.executeUpdate();
            prest.close();
            conn.commit();
            conn.setAutoCommit(true);
    }catch(SQLException e){
            e.printStackTrace();
    }
    }
}
```

HỦY MỘT GIAO DỊCH

- Trong quá trình thực hiện một giao dịch, nếu có sai sót nào xảy ra, ta có thể hủy giao dịch đang được thực hiện nửa chừng bằng cách sử dụng rollback
- conn.rollback() sẽ hủy dữ liệu đến thời điểm nó được commit gần nhất
  - Tùy thuộc vào hệ quản trị CSDL mà cách rollback có thể khác nhau
- Dữ liệu sau khi được commit sẽ không hủy được bằng rollback

VÍ DỤ rollback

```
public void rollbackTransaction(){
    String insertQuery = "INSERT INTO PRICELIST VALUES(?,?);";
    if (conn != null){
        try{    conn.setAutoCommit(false);
                PreparedStatement prest =
                        conn.prepareStatement(insertQuery);
                prest.setString(1, "Biscuit");
                prest.setDouble(2, 1.2);
                prest.executeUpdate();
                conn.commit();
                prest.setString(1, "Pen");
                prest.setDouble(2, 0.5);
                prest.executeUpdate();
                prest.close();
                conn.rollback();  conn.commit();
                conn.setAutoCommit(true);
        }catch(SQLException e){e.printStackTrace();}
    }
}
```

ResultSetMetaData

- *ResultSetMetaData* cung cấp các thông tin về cấu trúc cụ thể của *ResultSet*, bao gồm cả số cột, tên và giá trị của chúng



VÍ DỤ ResultSetMetaData

```
public void showMetadata(){
    String selectQuery = "SELECT * FROM PRICELIST;";
    if (conn != null){
        try{
            Statement st = conn.createStatement
                (ResultSet.TYPE_SCROLL_INSENSITIVE,
                                    ResultSet.CONCUR_READ_ONLY);
            ResultSet rs = st.executeQuery(selectQuery);
            ResultSetMetaData meta = rs.getMetaData();
            int iColumnCount = meta.getColumnCount();
            for (int i =1 ; i <= iColumnCount ; i++){
                System.out.println("Column Name: " + meta.getColumnName(i));
                System.out.println("Column Type: " + meta.getColumnTypeName(i));
                System.out.println("Display Size: " + meta.getColumnDisplaySize(i) );
                System.out.println("Precision: " + meta.getPrecision(i));
                System.out.println("Scale: " + meta.getScale(i) );

            }
            st.close();
        }catch(SQLException e){
                e.printStackTrace();

        }
    }
}
```

VÍ DỤ VỀ HIỂN THỊ MỘT TABLE

Ví dụ: Chương trình hiển thị một bảng dữ liệu.

```
import java.sql.*; import java.util.StringTokenizer;
public class TableViewer {
    final static String jdbcURL = "jdbc:odbc:StudentDB";
    final static String jdbcDriver = "sun.jdbc:odbc:JdbcOdbcDriver";
    final static String table = "STUDENT";
    public static void main(java.lang.String[]args) {
      System.out.println("---Table Viewer ---");
      try {  Class.forName(jdbcDriver);
            Connection con =
                        DriverManager.getConnection(jdbcURL, "", "");
            Statement stmt = con.createStatement();
            //  Đọc ra cả bảng Student  và đưa vào đối tượng rs
            ResultSet rs = stmt.executeQuery("SELECT * FROM " + table);
            //  Đọc ra các thông tin về rs
            ResultSetMetaData rsmd = rs.getMetaData();
            //  Xác định số cột của rsmd
            int colCount = rsmd.getColumnCount();
//xem tiếp ở slide tiếp theo
```

VÍ DỤ VỀ HIỂN THỊ MỘT TABLE

```
for(int col = 1; col <= colCount; col++)
{           //  In ra tên và kiểu của từng trường dữ liệu trong rsmd
            System.out.print(rsmd.getColumnLabel(col));
            System.out.print(" (" + rsmd.getColumnTypeName(col) + ")");
            if(col < colCount) System.out.print(", ");
}
System.out.println();
while(rs.next()){
            //  In ra dòng dữ liệu trong rsmd
            for(int col = 1; col <= colCount; col++)
            {
                        System.out.print(rs.getString(col));
                        if(col < colCount) System.out.print("  ");
            }
            System.out.println();
} //xem tiếp ở slide tiếp theo
```

VÍ DỤ VỀ HIỂN THỊ MỘT TABLE

```
            rs.close();
            stmt.close();
            con.close();
            }
            catch (ClassNotFoundException e) {
                        System.out.println("Unable to load database driver class");
            }
            catch (SQLException se) {
                        System.out.println("SQL Exception: " + se.getMessage());
            }
      }
}
```

ĐÓNG KẾT NỐI

- Để đóng kết nối ta sử dụng phương thức close của đối tượng connection:
  - connetion.close()



HẾT

BÀI 9